

BigchainDB uses the public key cryptography system for registering users and maintaining the integrity and security of the assets. Each asset has to be signed with the owner's private key. This provides surety that a transaction for the asset is being made by the owner of the asset and no one else. Additionally, each transaction generates its own unique hash called the transaction ID. Figure 1 illustrates a 'CREATE' transaction life cycle and Figure 2 illustrates a 'TRANSFER' transaction life cycle.

Creating and transferring an asset

Thus far, we have understood how BigchainDB works from a theoretical perspective. Now comes the fun part where we get to test our understanding with actual code. So let us continue with the example we have used in the previous section. Mike wants to register his painting as an asset and transfer its ownership to Sarah.

For starters, we need to decide what language we are going to use. BigchainDB provides drivers for all major

programming languages. For the purpose of this example, we shall be using Python.

Step 1: Install the Python driver for BigchainDB.

```
$ pip3 install bigchaindb_driver
```

Step 2: Now let's open up a new Python file and import the necessary modules.

```
from bigchaindb_driver import BigchainDB
from bigchaindb_driver.crypto import generate_keypair
from time import sleep
from sys import exit
```

Step 3: Register Mike and Sarah as users by generating their public/private key pair.

```
mike, sarah = generate_keypair(), generate_keypair()
```

Step 4: Set up the root URL. If you are running the node

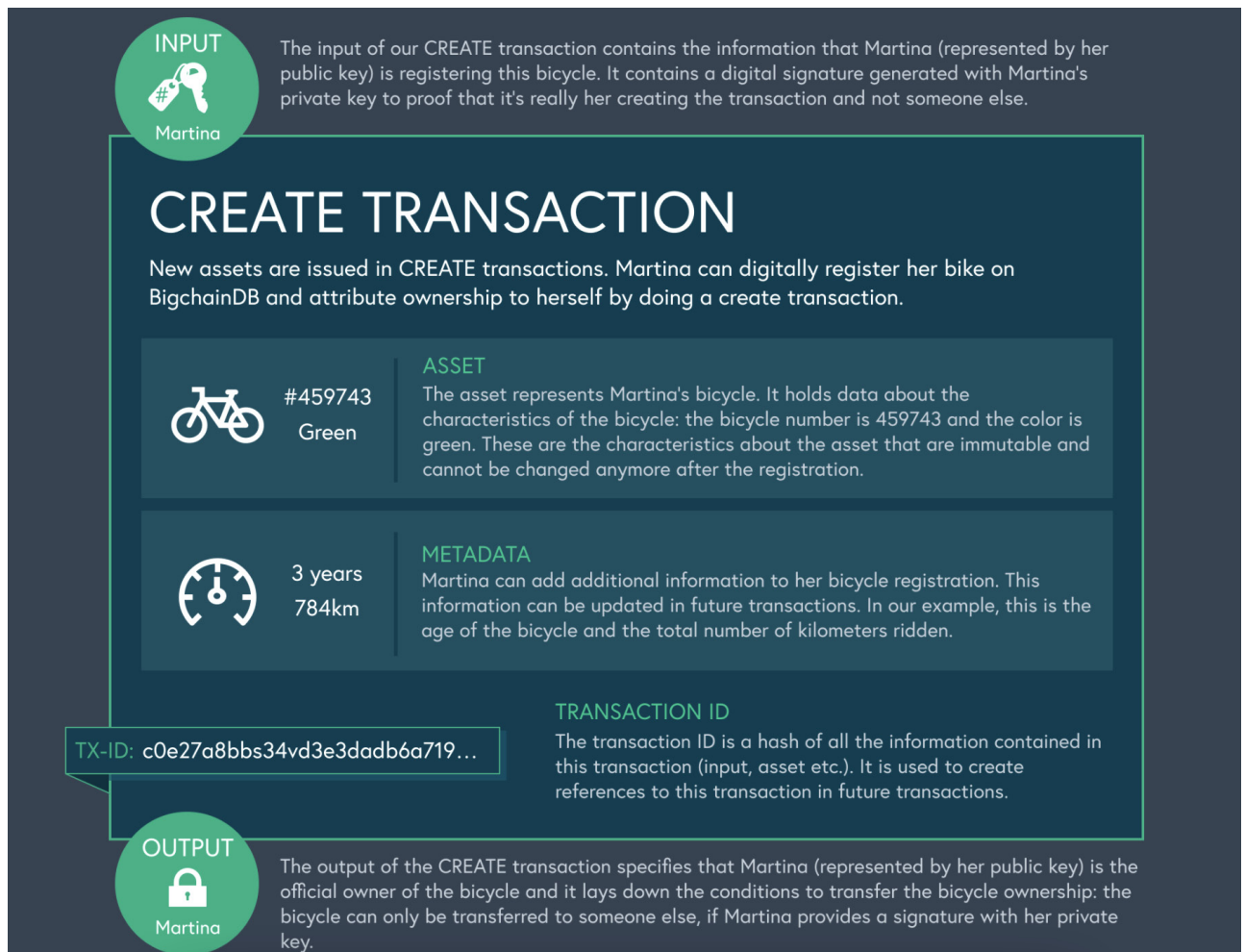


Figure 1: CREATE transaction in BigchainDB (Courtesy: BigchainDB)